

BRACT's
Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: E

Batch: 1

Practical No: 3

Roll no: 52

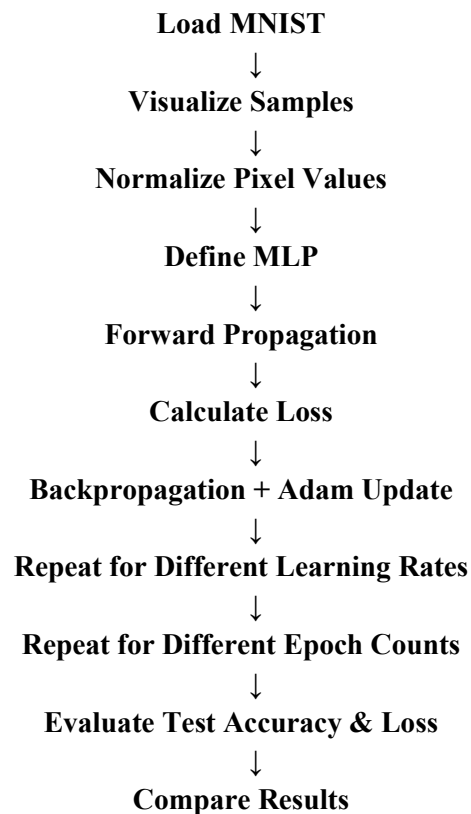
PRN No: 12410371

Name of Student: Vaidik Samarth

Subject Name: Deep Learning

Title of Practical: Implement forward propagation and backpropagation using TensorFlow/Keras. Analyse the effect of different learning rates and the number of epochs on model performance.

Methodology:



Code & Output:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Flatten
from tensorflow.keras.optimizers import Adam

print("Tensorflow version:", tf.__version__)

-- TensorFlow version: 2.20.0

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)

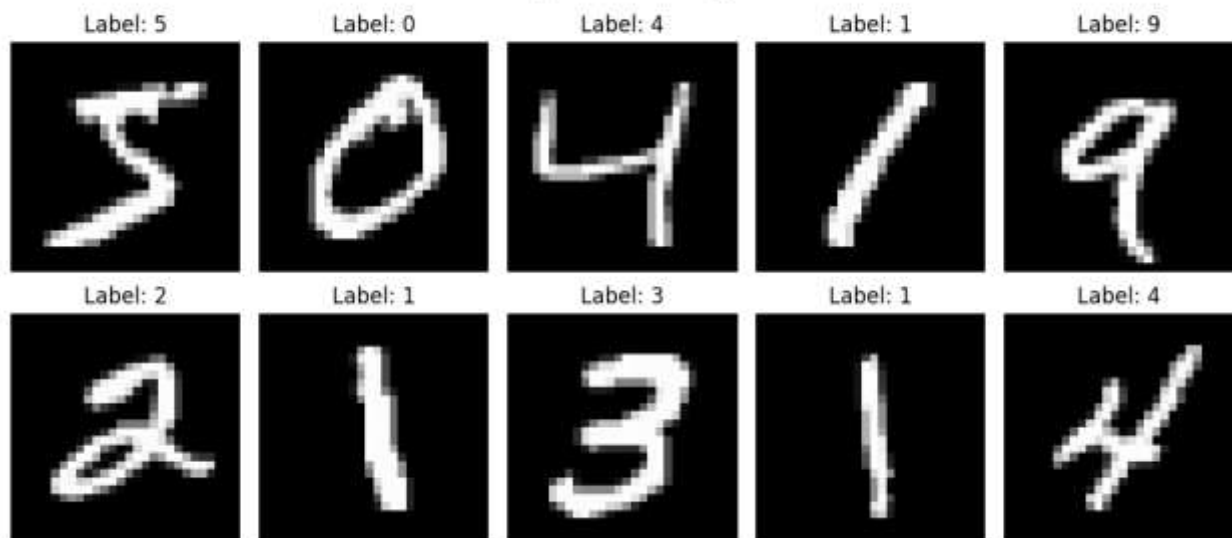
-- Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.epi:
11490434/11490434 ----- 0s 8us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.suptitle("Sample MNIST Images", fontsize=14)
plt.tight_layout()
plt.show()
```

Sample MNIST Images



```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Training pixel range:", x_train.min(), "to", x_train.max())
print("Testing pixel range:", x_test.min(), "to", x_test.max())

Training pixel range: 0.0 to 1.0
Testing pixel range: 0.0 to 1.0
```

```

def create_model(learning_rate):
    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation="relu"),
        Dense(64, activation="relu"),
        Dense(10, activation="softmax")
    ])

    optimizer = Adam(learning_rate=learning_rate)

    model.compile(
        optimizer=optimizer,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model

```

```

learning_rates = [0.1, 0.01, 0.001]
epochs_list = [5, 10, 20]

results = {}
histories = {}

for learning_rate in learning_rates:
    for epochs in epochs_list:
        print(f"~" * 60)
        print(f"Learning Rate: {learning_rate} | Epochs: {epochs}")

        # Initialize the model for every configuration
        tf.keras.backend.clear_session()
        model = create_model(learning_rate)

        history = model.fit(
            x_train,
            y_train,
            epochs=epochs,
            validation_data=(x_test, y_test),
            verbose=1
        )

        loss, accuracy = model.evaluate(x_test, y_test, verbose=0)

        results.append(
            {
                "learning_rate": learning_rate,
                "Epochs": epochs,
                "Accuracy": accuracy,
                "Loss": loss
            }
        )

        histories[(learning_rate, epochs)] = history.history

```

```

Learning Rate: 0.1 | Epochs: 5
Epoch 1/5
1875/1875 — 11s 5ms/step - accuracy: 0.1316 - loss: 2.3435 - val_accuracy: 0.0974 - val_loss: 2.3112
Epoch 2/5
1875/1875 — 8s 4ms/step - accuracy: 0.1030 - loss: 2.3132 - val_accuracy: 0.1135 - val_loss: 2.3107
Epoch 3/5
1875/1875 — 9s 5ms/step - accuracy: 0.1003 - loss: 2.3143 - val_accuracy: 0.1135 - val_loss: 2.3069
Epoch 4/5
1875/1875 — 10s 5ms/step - accuracy: 0.1030 - loss: 2.3138 - val_accuracy: 0.0980 - val_loss: 2.3191
Epoch 5/5
1875/1875 — 8s 5ms/step - accuracy: 0.1024 - loss: 2.3141 - val_accuracy: 0.0958 - val_loss: 2.3155

```

```

Learning Rate: 0.1 | Epochs: 10
Epoch 1/10
1875/1875 — 9s 4ms/step - accuracy: 0.3699 - loss: 1.7263 - val_accuracy: 0.3512 - val_loss: 1.5425
Epoch 2/10
1875/1875 — 10s 5ms/step - accuracy: 0.3378 - loss: 1.5745 - val_accuracy: 0.2912 - val_loss: 1.6254
Epoch 3/10
1875/1875 — 10s 5ms/step - accuracy: 0.2623 - loss: 1.7551 - val_accuracy: 0.2037 - val_loss: 1.8481
Epoch 4/10
1875/1875 — 10s 5ms/step - accuracy: 0.1892 - loss: 1.9032 - val_accuracy: 0.1902 - val_loss: 1.9038
Epoch 5/10
1875/1875 — 9s 5ms/step - accuracy: 0.1922 - loss: 1.8674 - val_accuracy: 0.2084 - val_loss: 1.8276
Epoch 6/10
1875/1875 — 9s 5ms/step - accuracy: 0.1971 - loss: 1.8501 - val_accuracy: 0.1937 - val_loss: 1.8937
Epoch 7/10
1875/1875 — 11s 6ms/step - accuracy: 0.1935 - loss: 1.8886 - val_accuracy: 0.1935 - val_loss: 1.9287
Epoch 8/10
1875/1875 — 19s 5ms/step - accuracy: 0.1918 - loss: 1.9044 - val_accuracy: 0.1851 - val_loss: 1.9239
Epoch 9/10
1875/1875 — 10s 5ms/step - accuracy: 0.1959 - loss: 1.8872 - val_accuracy: 0.1880 - val_loss: 1.8980
Epoch 10/10
1875/1875 — 9s 5ms/step - accuracy: 0.1955 - loss: 1.8800 - val_accuracy: 0.2005 - val_loss: 1.8936

```

```

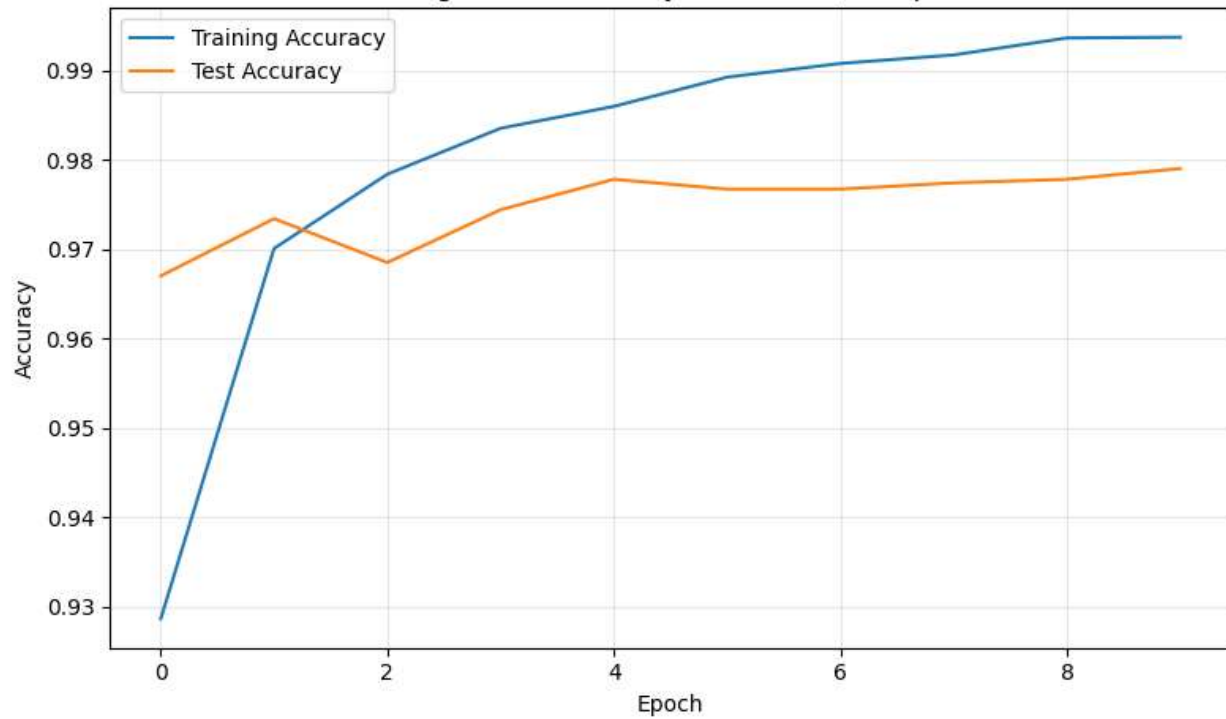
results_df = pd.DataFrame(results)
results_df["Accuracy"] = results_df["Accuracy"].round(4)
results_df["Loss"] = results_df["Loss"].round(4)

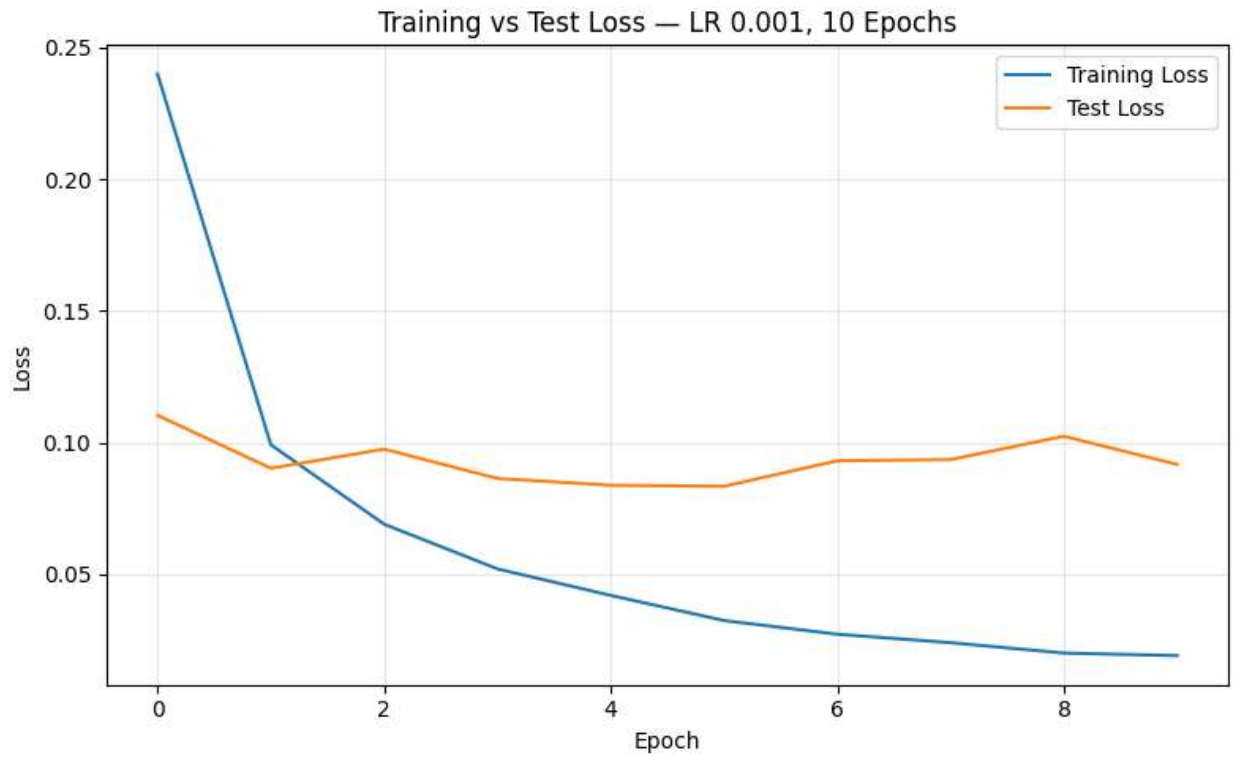
display(results_df)

```

	Learning Rate	Epochs	Accuracy	Loss
0	0.100	5	0.0958	2.3155
1	0.100	10	0.2005	1.8936
2	0.100	20	0.2069	1.8754
3	0.010	5	0.9545	0.1849
4	0.010	10	0.9705	0.1532
5	0.010	20	0.9676	0.2032
6	0.001	5	0.9759	0.0851
7	0.001	10	0.9790	0.0917
8	0.001	20	0.9795	0.1145

Training vs Test Accuracy — LR 0.001, 10 Epochs



**Conclusion:**

Forward propagation and backpropagation were implemented through TensorFlow/Keras model training. The experiment demonstrated that learning-rate selection strongly affects optimization and classification performance. Among the tested configurations in the submitted run, learning rate = 0.001 and epochs = 10 gave the highest test accuracy of 96.93%.